

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой №43
профессор, д-р техн. наук,
профессор

должность, уч. степень, звание

подпись, дата

М.Ю. Охтилев

инициалы, фамилия

БАКАЛАВРСКАЯ РАБОТА

на тему Разработка программного комплекса для автоматизации процессов согласования документов

выполнена Гусевым Иваном Евгеньевичем

фамилия, имя, отчество студента в творительном падеже

по направлению подготовки

09.03.04

код

Программная инженерия

наименование направления

направленности

наименование направления

02

код

Проектирование программных систем

наименование направленности

наименование направленности

Студент группы №

4132

подпись, дата

И. Е. Гусев

инициалы, фамилия

Руководитель

канд. техн. наук

должность, уч. степень, звание

подпись, дата

А. В. Фомин

инициалы, фамилия

Санкт-Петербург 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИВОРОСТРОЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой №43

д-р техн. наук, профессор

должность, уч. степень, звание

подпись, дата

М.Ю.Охтилев

инициалы, фамилия

ЗАДАНИЕ НА ВЫПОЛНЕНИЕ БАКАЛАВРСКОЙ РАБОТЫ

студенту группы

4132

номер

Гусеву Ивану Евгеньевичу

фамилия, имя, отчество

на тему Разработка программного комплекса для автоматизации процессов согласования документов

утвержденную приказом ГУАП от 28.03.2025 № 11-387/25

Цель работы: разработка программного комплекса, обеспечивающего повышение эффективности и безопасности внутреннего документооборота, а также упрощение процессов согласования и контроля исполнения.

Задачи, подлежащие решению: 1) проанализировать предметную область;
2) формализовать задачу и разработать архитектуру программного комплекса
3) создание программной реализации; 4) провести тестирование и отладку системы.

Содержание работы (основные разделы): введение, анализ предметной области,
постановка задачи и методология, проектирование системы, разработка и тестирование,
заключение

Срок сдачи работы « 06 » июня 2025

Руководитель

канд. техн. наук

должность, уч. степень, звание

31.03.2025

подпись, дата

А. В. Фомин

инициалы, фамилия

Задание принял(а) к исполнению

студент группы №

4132

31.03.2025

подпись, дата

И. Е. Гусев

инициалы, фамилия

Оглавление

Введение.....	5
1 Анализ предметной области.....	7
1.1 Основные понятия.....	7
1.2 Типовой процесс согласования документов.....	7
1.3 Проблемы ручного согласования документов.....	8
1.4 Анализ существующих решений.....	9
1.4.1 Коммерческие зарубежные системы.....	9
1.4.1.1 DocuSign.....	9
1.4.1.2 Microsoft SharePoint.....	9
1.4.2 Российские коммерческие решения.....	10
1.4.2.1 СБИС.....	10
1.4.2.2 1С:Документооборот.....	10
1.4.3 Решения с открытым исходным кодом.....	11
1.4.3.1 Alfresco Community Edition.....	11
1.4.3.2 Mayan EDMS.....	11
1.5 Сравнительный анализ.....	11
2 Постановка задачи и методология.....	13
2.1 Назначение программного обеспечения.....	13
2.2 Область применения.....	13
2.3 Цели разработки.....	13
2.3 Обоснование актуальности.....	13
2.4 Основные задачи.....	14
2.5 Целевая аудитория и пользователи системы.....	14
2.5.1 Инициатор согласования.....	15
2.5.2 Согласующий.....	15
2.5.3 Администратор системы.....	15
2.5.4 Наблюдатель.....	16
3 Проектирование системы.....	17
3.1 Характеристика и структура программы.....	17
3.2 Описание основных особенностей программы.....	18
3.3 Контекстная диаграмма IDEF0.....	19
3.4 Диаграмма использования.....	19
3.5 Описание основных сущностей.....	20
4 Требования к автоматизированной системе.....	22
4.1 Требования к структуре АС в целом.....	22
4.1.2 Перечень подсистем, их назначение и основные характеристики.....	22
4.1.3 Требования к способам и средствам обеспечения информационного взаимодействия компонентов АС.....	23
4.2 Требования к функциям графического интерфейса.....	23
4.2.1 Функции общей работы.....	23
4.2.2 Функции страницы входа в аккаунт.....	23
4.2.3 Функции панели управления администратора.....	24
4.2.4 Функции панели управления сотрудниками.....	26
4.2.5 Функции панели редактирования данных сотрудника.....	27
4.2.6 Функционал панели управления группами.....	28
4.2.7 Функционал панели редактирования группы.....	28
4.2.8 Функционал панели управления маршрутами.....	30
4.2.9 Функционал панели редактирования маршрута.....	30

4.2.10 Функциональные требования панели управления документами.....	32
4.2.11 Функциональные требования панели редактирования состояния документа.....	33
4.3 Нефункциональные требования.....	34
4.3.1 Производительность.....	34
4.3.2 Безопасность.....	34
4.3.4 Масштабируемость.....	34
4.3.4 Технические требования.....	34
4.3.4.1 Клиентская часть.....	34
4.3.4.2 Серверная часть.....	35
4.3.4.3 База данных.....	35
5 Разработка и тестирование.....	36
5.1 Среда разработки.....	36
5.2 Схема проекта.....	36
5.3 База данных.....	36
5.4 Структура серверной части.....	38
5.5 Реализованные контроллеры.....	39
5.6 Используемые технологии.....	39
5.7 Шифрование файлов.....	40
5.8 Развертывание.....	40
5.9 Клиентская часть.....	41
5.10 Тестирование.....	42
5.10.1 Ключевые аспекты.....	42
5.10.2 Проверяемые сценарии.....	42
5.10.3 Итоги тестирования.....	43
Заключение.....	44
Список использованных источников.....	45
Приложение А. Скрипт формирования базы данных.....	47
Приложение Б. Получение и шифрование файлов.....	51
Приложение В. Аутентификация с JWT.....	59
Приложение Г. Основные сущности.....	66
Приложение Д. Конфигурация docker.....	72

Введение

В эпоху цифровой трансформации эффективное управление документами становится стратегическим преимуществом для организаций любого масштаба. Однако существующие проприетарные системы документооборота часто оказываются слишком дорогими, недостаточно гибкими или не соответствующими специфическим требованиям отдельных предприятий. Это создает значительный спрос на открытые решения, которые могли бы служить базой для создания специализированных систем согласования документов [1].

История развития open-source движения демонстрирует, что наиболее успешные проекты сочетают в себе:

- модульную архитектуру, позволяющую легко адаптировать систему под конкретные нужды
- открытый код, обеспечивающий прозрачность и безопасность решения
- активное сообщество разработчиков, способствующее постоянному улучшению продукта

Актуальность данного проекта обусловлена тремя ключевыми факторами:

- Необходимость независимости от экономической и политической ситуации в мире — решение останется пригодным для использования вне зависимости от глобальной политической и экономической обстановки [2].
- Потребностью в кастомизации - каждая организация имеет уникальные бизнес-процессы, требующие индивидуальных настроек
- Экономической эффективностью - open-source решение позволяет значительно сократить затраты на внедрение и развитие системы

Основной целью работы является создание open-source платформы для автоматизации согласования документов, которая:

- предоставляет базовый функционал для большинства стандартных сценариев
- имеет четко определенные API для расширения возможностей

Объектом исследования выступают open-source системы документооборота, а предметом исследования - методы их адаптации под различные бизнес-процессы.

Для достижения поставленной цели необходимо решить следующие задачи:

- Проанализировать существующие open-source аналоги и выявить лучшие практики
- Разработать модульную архитектуру с возможностью расширения функционала
- Реализовать ядро системы с базовыми возможностями:
 - управление маршрутами согласования
 - сохранение предыдущих версий документов
 - просмотр доступных документов
- Обеспечить простой механизм развертывания

Научная новизна проекта заключается в:

- разработке универсальной модели маршрутизации документов
- создании механизма плагинов для расширения функциональности
- реализации open API для интеграции с внешними системами

Практическая значимость:

- Для малого бизнеса - готовое бесплатное решение
- Для крупных предприятий - основа для создания корпоративных систем
- Для разработчиков - возможность создавать специализированные модули

Работа состоит из трех основных глав:

- Аналитической - исследование предметной области и существующих решений
- Проектной - разработка архитектуры и API системы
- Практической - реализация ядра системы и тестирование

Данный проект создает основу для развития экосистемы решений в области автоматизации документооборота, позволяя организациям и разработчикам сосредоточиться на создании специализированных модулей вместо разработки базовой функциональности с нуля.

1 Анализ предметной области

1.1 Основные понятия

В ходе работы будут использоваться следующие понятия:

1. Документооборот – система создания, обработки, хранения и контроля документов в организации [1].
2. Согласование документа – процесс проверки и утверждения документа ответственными лицами.
3. Маршрут согласования – последовательность этапов, которые проходит документ до финального утверждения.
4. Юридически значимый электронный документ - документ, подписанный квалифицированной электронной подписью (КЭП).

1.2 Типовой процесс согласования документов

Современные организации осуществляют оборот документов различного содержания и назначения: договоры, приказы, регламенты, финансовые отчеты, техническая документация и другие. Доступ к этим документам и права на их редактирование строго регламентируются в соответствии с должностными обязанностями сотрудников и внутренними политиками безопасности организации. В зависимости от сферы деятельности организации (коммерческая, государственная, образовательная и др.) процесс документооборота может дополнительно регулироваться:

- Уставом и внутренними регламентами организации
- Федеральными законами (например, ФЗ-152 "О персональных данных") [8]
- Отраслевыми стандартами и ГОСТами [6, 7]
- Международными стандартами (для компаний с иностранным участием) [9]

Основные этапы жизненного цикла документа в Таблице 1 [3].

Название этапа	Описание
Формирование маршрута	На данном этапе руководитель или администратор документооборота описывает формат документа, этапы согласования, а также какие группы лиц имеют на каждом конкретном этапе доступ к документу.
Инициализация процесса	Сотрудник выбирает маршрут документа,

	загружает его в систему, запускает процесс согласования
Согласование и правки	На каждом конкретном этапе сотрудники, имеющие доступ к документу, производят его правки, а затем информируют участников следующего этапа о завершении.
Утверждение	Документ завершает маршрут, получает статус «утвержден», сохраняется в системе.

Таблица 1 - Основные этапы жизненного цикла документа.

1.3 Проблемы ручного согласования документов

Ручное согласование документов, осуществляемое либо через бумажные носители, либо через общие сетевые папки, имеет ряд существенных недостатков, описанных в Таблице 2 [5].

Название проблемы	Описание проблемы
Сохранность и целостность документа	Сотрудники могут утратить бумажную версию документа или удалить из системы электронную версию без возможности восстановления. В ходе многочисленных правок может быть потеряна важная информация как и изначальная версия документа.
Контроль доступа	Возникают сложности отслеживания работы лиц, имеющих доступ к документу, а также защиты от несанкционированного доступа.
Проблемы перехода на следующий этап	Передача документа на следующий этап согласования подразумевает информирования группы лиц, ответственных за следующий этап, напрямую или за счет других средств, что подразумевает наличия у сотрудника знаний обо всех участниках процесса. Информирование таким образом может быть временно затратным.
Контроль статуса документа	Нет способа следить за статусом кроме как через сотрудников и ручные пометки.

Таблица 2 - Основные проблемы ручного согласования

1.4 Анализ существующих решений

В условиях цифровизации бизнес-процессов на рынке представлено множество систем для автоматизации документооборота – как коммерческих, так и open-source. Однако при детальном анализе выявляются существенные ограничения существующих решений: отсутствие полной поддержки российских стандартов, высокая стоимость владения, сложность интеграции или недостаточная гибкость [11, 14].

В данном разделе проводится сравнительный анализ ключевых платформ по следующим критериям:

- Возможности настройки маршрутов согласования
- Экономическая эффективность внедрения
- Гибкость архитектуры и простота интеграции

1.4.1 Коммерческие зарубежные системы

1.4.1.1 DocuSign

Преимущества:

- Интуитивный интерфейс и мобильные приложения
- Поддержка 30+ форматов документов
- Интеграция с Salesforce, Google Workspace, Office 365

Недостатки:

- Отсутствие полноценной поддержки российских КЭП (только API для интеграции)
- Высокая стоимость
- Хранение данных в зарубежных дата-центрах (проблемы с 152-ФЗ)
- Невозможность глубокой кастомизации

1.4.1.2 Microsoft SharePoint

Преимущества:

- Глубокая интеграция с продуктами Microsoft
- Мощные инструменты совместной разработки
- Гибкая система разрешений

Недостатки:

- Требуется лицензий Microsoft 365 (от \$12.5/пользователь/месяц)
- Сложность администрирования
- Высокие требования к инфраструктуре

1.4.2 Российские коммерческие решения

1.4.2.1 СБИС

Преимущества:

- Полное соответствие российскому законодательству
- Встроенная поддержка КЭП и НЭП
- Интеграция с государственными системами (ФНС, ПФР)
- Локализованная техподдержка

Недостатки:

- Закрытая проприетарная архитектура
- Высокая стоимость внедрения (от 500 тыс. руб.)
- Жесткие требования к серверному оборудованию
- Ограниченные возможности кастомизации

1.4.2.2 1С:Документооборот

Преимущества:

- Готовая интеграция с 1С:Предприятие
- Поддержка отраслевых регламентов
- Встроенные шаблоны документов
- Доступная цена (от 75 тыс. руб. лицензия)

Недостатки:

- Низкая производительность при больших объемах
- Устаревший пользовательский интерфейс
- Слабая поддержка мобильных устройств
- Ограниченные API для интеграции

1.4.3 Решения с открытым исходным кодом

1.4.3.1 Alfresco Community Edition

Преимущества:

- Модульная ECM-платформа
- Поддержка стандарта CMIS
- Гибкая система workflow
- Возможности машинного обучения

Недостатки:

- Отсутствие готовой поддержки российских ЭП
- Необходимость доработки под российские стандарты
- Высокие требования к серверным ресурсам

1.4.3.2 Mayan EDMS

Преимущества:

- Простота развертывания (Python/Django)
- Открытая лицензия (Apache 2.0)
- Активное сообщество разработчиков

Недостатки:

- Ограниченные возможности маршрутизации
- Нет встроенной поддержки workflow
- Слабая производительность при >1000 документов/день
- Минимальная документация на русском языке

1.5 Сравнительный анализ

Проведенный анализ рынка систем автоматизации документооборота выявил существенный технологический разрыв между существующими решениями и современными требованиями организаций. Коммерческие продукты, несмотря на развитую функциональность, остаются дорогостоящими "черными ящиками" с ограниченными возможностями адаптации под специфические бизнес-процессы. Open-source альтернативы, в свою очередь, требуют значительных доработок для

промышленного использования и не предлагают готовых механизмов для реализации сложных маршрутов.

Ключевые болевые точки текущих решений:

- Архитектурная ригидность - большинство систем построены как монолиты, что затрудняет их масштабирование и интеграцию
- Экономическая неэффективность - совокупная стоимость владения коммерческими решениями превышает разумные пределы для среднего бизнеса
- Технологическая устарелость - многие платформы используют устаревающие подходы к хранению и обработке документов
- Закрытость экосистем - проприетарные форматы данных и ограниченные API создают проблемы при кастомизации решений

2 Постановка задачи и методология

2.1 Назначение программного обеспечения

Разрабатываемое программное обеспечение (далее — автоматизированная система(АС)) предназначено для автоматизации процессов согласования документов в организациях различного масштаба. Система обеспечивает формирование маршрутов согласования, отслеживание статусов, управление версиями документов, уведомление участников, контроль прав доступа и формирование отчетности по документам. Это позволяет повысить прозрачность, управляемость и скорость согласования документов, а также снизить количество ошибок, связанных с человеческим фактором.

2.2 Область применения

Программный комплекс может использоваться в коммерческих, государственных и образовательных организациях, где существуют процедуры согласования внутренних или внешних документов: приказов, договоров, заявок, служебных записок и других типов управленческой документации. Система может быть внедрена как самостоятельное приложение, так и интегрирована в существующую информационную инфраструктуру предприятия.

2.3 Цели разработки

Целью разработки является создание надежного и гибкого программного комплекса, обеспечивающего:

- автоматизацию согласования документов с возможностью настройки маршрутов;
- прозрачность процессов за счёт сохранения истории изменений и версии документов;
- минимизацию времени, затраченного на согласование;
- повышение информационной безопасности за счёт разграничения прав доступа.

2.3 Обоснование актуальности

1. Рост требований к эффективности и прозрачности бизнес-процессов. В современных организациях скорость и прозрачность согласования документов напрямую влияют на управляемость и конкурентоспособность. Ручные и полуавтоматизированные процессы приводят к задержкам, ошибкам и усложняют контроль над документооборотом [2].

2. Высокая стоимость и ограниченная гибкость коммерческих решений. Большинство проприетарных систем автоматизации документооборота требуют значительных финансовых вложений на этапе внедрения и сопровождения, а также часто не позволяют гибко адаптировать функционал под уникальные бизнес-процессы конкретной организации .
3. Необходимость независимости от внешних факторов. В условиях санкционного давления и изменений на мировом рынке критически важно иметь решения, не зависящие от зарубежных поставщиков и экономической ситуации. Open-source платформа обеспечивает независимость и возможность локализации под российские стандарты .
4. Потребность в кастомизации и расширяемости. Каждая организация имеет свои особенности документооборота, и универсальные решения редко полностью соответствуют этим требованиям. Открытая архитектура и модульность open-source систем позволяют быстро вносить изменения и расширять функциональность
5. Экономическая эффективность open-source подхода. Использование открытого программного обеспечения позволяет существенно снизить совокупную стоимость владения системой, а также уменьшить затраты на развитие и поддержку за счёт активного сообщества разработчиков .

2.4 Основные задачи

Для достижения поставленной цели в рамках разработки системы необходимо решить следующие задачи:

- Реализовать механизм создания и редактирования маршрутов согласования.
- Обеспечить поддержку различных типов согласования: последовательного, параллельного, смешанного.
- Внедрить систему уведомлений и напоминаний участникам процесса.
- Организовать контроль версий документов с сохранением истории изменений.
- Внедрить систему ролевого доступа с разграничением прав.
- Обеспечить интерфейс для взаимодействия с системой

2.5 Целевая аудитория и пользователи системы

Программный комплекс ориентирован на использование в организациях, где требуется согласование различных типов документов, включая внутренние распоряжения,

приказы, договора, заявки, отчеты и другие документы, требующие прохождения утверждающих этапов.

2.5.1 Инициатор согласования

Сотрудник, который создает новый документ и запускает процесс согласования.

Основные действия в системе:

- Создание и загрузка документа
- Определение последовательности и участников согласования.
- Контроль текущего состояния процесса.
- Получение информации о результатах рассмотрения.
- Возможность повторного направления документа на согласование после корректировки.

2.5.2 Согласующий

Пользователь, участвующий в рассмотрении документа на одном из этапов согласования. Получает документ для ознакомления и принятия решения: подтверждает согласование, отклоняет или отправляет на доработку.

Основные действия в системе:

- Просмотр поступивших на согласование документов;
- Принятие решения: согласовать, отклонить, направить на доработку;
- Оставление комментариев и замечаний;
- Передача документа следующему участнику маршрута (при необходимости);
- Получение уведомлений о поступлении нового документа.

2.5.3 Администратор системы

Пользователь с полным доступом к настройкам системы, администрированию пользователей и маршрутов.

Основные действия в системе:

- Управление учетными записями пользователей и ролями;
- Настройка маршрутов согласования (шаблоны);

- Контроль целостности данных и журналов действий;
- Мониторинг активных процессов и диагностика проблем;
- Управление параметрами безопасности и доступом к данным;

2.5.4 Наблюдатель

Пользователь, который имеет право наблюдать за процессом согласования, но не участвует в принятии решений.

Основные действия в системе:

- Просмотр статусов текущих процессов;
- Доступ к отчетам и истории согласований;
- Получение уведомлений о завершении согласований (например, для руководства).

3 Проектирование системы

3.1 Характеристика и структура программы

В программный комплекс, предназначенный для автоматизации процессов согласования документов, входят следующие ключевые функциональные компоненты:

- Модуль аутентификации и авторизации
- Модуль управления пользователями
- Модуль управления группами
- Модуль распределения ролей и управления доступом
- Модуль маршрутизации и согласования документов
- Модуль версионирования документов
- Модуль хранения и предоставления доступа к данным

Модуль аутентификации и авторизации осуществляет идентификацию пользователей при входе в систему. Он поддерживает регистрацию новых аккаунтов (для администраторов), хранит пароли в зашифрованном виде и назначает соответствующие роли. Также реализует контроль доступа к функциям системы в зависимости от статуса пользователя (инициатор, согласующий, администратор).

Модуль управления пользователями предоставляет интерфейс и API для добавления, изменения и удаления учетных записей сотрудников. С его помощью можно просматривать список пользователей, осуществлять поиск по ФИО и должности, а также группировать сотрудников по отделам или проектам.

Модуль управления группами предназначен для объединения пользователей в логические группы, что облегчает маршрутизацию согласования и распределение прав.

Модуль распределения ролей и управления доступом отвечает за разграничение прав пользователей при работе с функционалом и данными системы. Он определяет, какие действия разрешены каждой роли: просмотр, согласование, редактирование, администрирование.

Модуль маршрутизации и согласования документов реализует настройку маршрутов согласования, включая последовательные, параллельные и условные схемы. Документы автоматически направляются по заданным сценариям, при этом система отслеживает их статусы и сохраняет историю согласований.

Модуль уведомлений и напоминаний информирует пользователей о необходимости согласования, появлении новых документов, завершении маршрутов или возникновении

ошибок. Оповещения могут отправляться как через интерфейс системы, так и по электронной почте. Возможна настройка по типам событий и ролям.

Модуль версионирования документов фиксирует все изменения, внесённые в документы на этапе согласования. Пользователи могут просматривать историю версий, сравнивать изменения и при необходимости восстанавливать предыдущие редакции.

Модуль отчетности позволяет формировать разнообразные отчеты по активности пользователей, времени прохождения маршрутов, количеству согласованных или отклонённых документов и другим параметрам. Данные могут экспортироваться в форматы PDF, Excel и docx.

Модуль хранения и предоставления доступа к данным обеспечивает взаимодействие с базой данных (PostgreSQL), отвечает за сохранность и целостность информации. Он реализует выполнение операций создания, чтения, обновления и удаления таких сущностей, как пользователи, документы, маршруты, группы

3.2 Описание основных особенностей программы

Данная программа представляет собой комплексное решение для автоматизации процессов согласования документов в корпоративной среде. Система обладает высокой модульностью, что позволяет разделить функционал на отдельные блоки, каждый из которых выполняет свою конкретную задачу — от авторизации пользователей до управления уведомлениями и работы с хранилищем файлов. Такой подход облегчает сопровождение, масштабирование и адаптацию системы под изменяющиеся требования бизнеса.

Для хранения документов используется Samba-сервер, который обеспечивает надежное распределённое хранение файлов. Это позволяет обеспечить кроссплатформенный доступ к документам и гибко управлять правами пользователей. Важной особенностью является поддержка версий документов — все изменения в документах фиксируются и сохраняются, что гарантирует прозрачность и возможность восстановления предыдущих редакций.

Администратор может добавлять, редактировать и удалять пользователей, объединять их в группы для удобного управления маршрутами. Доступ к документам регулируется ролями с разными правами — это повышает безопасность при работе с файлами на Samba-сервере.

Маршруты согласования можно настраивать под разные сценарии: последовательно, параллельно или комбинируя этапы. Система поддерживает условия

перехода между этапами и автоматические действия, например, при отклонении документа или задержке на согласовании. Такой подход помогает ускорить процессы и снизить задержки.

Интегрированная система уведомлений и напоминаний информирует пользователей о новых документах, необходимости согласования, завершении маршрутов, а также о возможных ошибках. Уведомления могут поступать как через интерфейс программы, так и по электронной почте, что помогает ускорить обработку документов и снизить вероятность пропуска важных событий.

Контроль версий документов обеспечивает сохранение полной истории изменений, что позволяет отслеживать внесённые правки и при необходимости восстанавливать предыдущие версии. Все действия пользователей фиксируются в системе, что обеспечивает возможность аудита и повышает прозрачность работы.

3.3 Контекстная диаграмма IDEF0

Диаграммы IDEF0 позволяют структурировать и описать бизнес-процессы, определяя входы, выходы, управляющие воздействия и механизмы для каждой из функций, что способствует лучшему пониманию логики работы всей системы.



Рисунок 1 - IDEF0

3.4 Диаграмма использования

Диаграмма использования (Use Case Diagram) используется для отображения взаимодействия пользователей (актеров) с системой. Она позволяет описать основные

сценарии использования, показывая, какие функции доступны разным категориям пользователей, и помогает определить границы системы и её ключевые требования.

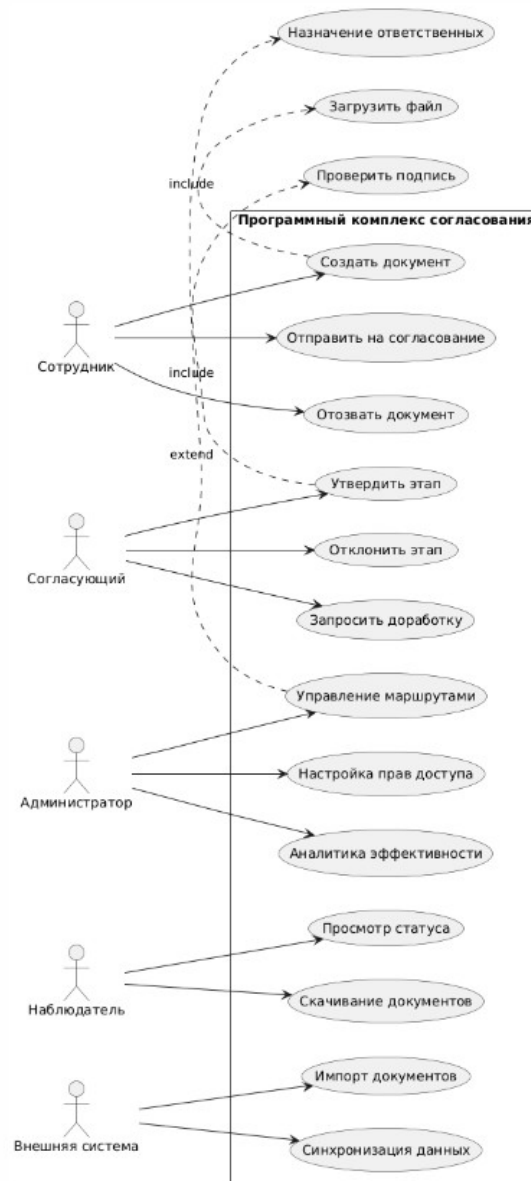


Рисунок 2 — Диаграмма использования

3.5 Описание основных сущностей

В проекте будут использоваться следующие сущности:

- **Сотрудник (User).** Пользователь системы — может быть инициатором документа, участником маршрута согласования или исполнителем. Содержит идентификатор, уникальный логин, хэшированный пароль, имя, электронную почту, должность, подразделение и список групп, в которые входит пользователь.

- Группа пользователей (UserGroup). Объединяет сотрудников для упрощения назначения прав. Содержит идентификатор, название, описание группы и список пользователей, которые в неё входят.
- Исполнитель (Executor). Определяет, кто может работать с документом — конкретный пользователь или целая группа. Содержит идентификатор, ссылку на пользователя (если исполнитель — один человек) или на группу (если исполнитель — группа).
- Документ (Document). Основная сущность системы, проходящая согласование. Содержит идентификатор, название, описание, секретный ключ для доступа, статус (черновик, в процессе, согласован и т. д.), текущий этап согласования, исполнителя, инициатора, зарезервировавшего пользователя, маршрут, список версий, даты создания и обновления.
- Маршрут согласования (Route). Определяет последовательность этапов, которые проходит документ. Содержит идентификатор и упорядоченный список этапов.
- Этап маршрута (RouteNode). Конкретный шаг в процессе согласования. Содержит идентификатор, ссылку на маршрут, исполнителя, порядковый номер и описание этапа.
- Версия документа (Version). Хранит файлы документа и их историю. Содержит идентификатор, ссылку на документ, путь к файлу и контрольную сумму для проверки целостности.
- Статус документа (DocumentStatus). Перечисление возможных состояний документа: в процессе (IN_PROGRESS), согласован (APPROVED), отклонён (REJECTED), исполнен (COMPLETED), в архиве (ARCHIVED).

4 Требования к автоматизированной системе

4.1 Требования к структуре АС в целом

4.1.2 Перечень подсистем, их назначение и основные характеристики

Автоматизированная система согласования документов включает в себя следующие подсистемы:

1. Модуль управления документами. Обеспечивает загрузку, хранение, просмотр и редактирование документов. Поддерживает работу с версиями, сохраняет историю изменений и позволяет восстанавливать предыдущие версии документа.
2. Модуль маршрутов согласования. Предоставляет инструменты для настройки, запуска и отслеживания маршрутов согласования. Поддерживает как линейное (последовательное), так и параллельное согласование, а также возможность динамического изменения маршрута.
3. Модуль уведомлений и напоминаний. Автоматически формирует и отправляет пользователям уведомления о новых задачах на согласование, приближении сроков, завершении процессов и других событиях.
4. Модуль управления ролями и доступом. Обеспечивает контроль доступа к функциональности и данным системы. Реализует разграничение прав пользователей по ролям: инициатор, согласующий, администратор и наблюдатель.
5. Модуль сеансов и аутентификации. Обеспечивает вход пользователей в систему, создание пользовательских сессий, проверку подлинности и авторизацию. Поддерживается механизм хранения сессий, а также автоматическое завершение сессии по тайм-ауту.
6. Модуль взаимодействия с базой данных. Осуществляет подключение к реляционной базе данных. Выполняет операции по чтению, записи и обновлению данных о документах, маршрутах, пользователях, логах и отчетах. Обеспечивает отказоустойчивость и сохранность данных.
7. Пользовательский интерфейс. Предназначен для работы конечных пользователей системы. Обеспечивает доступ к функциональности в соответствии с ролью пользователя. Интерфейс построен на базе UI-фреймворка и взаимодействует с backend через REST API.

4.1.3 Требования к способам и средствам обеспечения информационного взаимодействия компонентов АС

Для обеспечения надежного и эффективного обмена информацией между различными компонентами автоматизированной системы согласования документов применяется клиент-серверная архитектура. Вся основная логика обработки данных и их хранения сосредоточена на серверной стороне, которая построена на платформе Java с использованием популярного фреймворка Spring Framework. Сервер взаимодействует с системой управления базами данных PostgreSQL посредством ORM-технологий, таких как Hibernate и JPA, что упрощает работу с данными и повышает производительность.

Клиентская часть представлена веб-интерфейсом, доступным через любой современный браузер. Для создания удобного и интерактивного пользовательского интерфейса применяются современные UI-фреймворки, а связь с сервером осуществляется через REST API, что обеспечивает гибкий и масштабируемый обмен данными.

Передача информации между клиентом и сервером происходит по защищённому протоколу HTTPS, что гарантирует безопасность и конфиденциальность передаваемых данных. Все элементы системы — фронтенд, бэкенд и база данных — развёрнуты либо внутри защищённой корпоративной сети, либо в изолированной облачной среде, что дополнительно повышает уровень безопасности и контроля над инфраструктурой.

4.2 Требования к функциям графического интерфейса

4.2.1 Функции общей работы

- возможность входа пользователя в систему (с авторизацией по логину и паролю);
- возможность регистрации нового пользователя (администратор);
- возможность просмотра всех документов, к которым у пользователя есть доступ;
- возможность поиска документов по фильтрам (дата, статус, инициатор, название);
- возможность редактирования личного профиля;
- возможность получения уведомлений

4.2.2 Функции страницы входа в аккаунт

Авторизация пользователя в системе производится через следующую форму.

Вход

Логин

Пароль 

Рисунок 4 – Форма входа в аккаунт

В поле «Логин» должны быть введены данные строкового типа. В данное поле записывается имя пользователя.

В поле «Пароль» должны быть введены данные строкового типа. Пароль вводится в состоянии маски (маска имеет формат *). При нажатии на знак показа пароля, пароль показывается пользователю. При повторном нажатии пароль переходит в состояние маски. При нажатии кнопки «Войти» происходит проверка логина и пароля. В случае, если аккаунт с данным логином и паролем существует, происходит переход на главную страницу.

4.2.3 Функции панели управления администратора

Форма для предоставления доступа к функционалу администратора для мониторинга и управления:

- Уведомления
- Маршруты
- Документы
- Сотрудники и группы

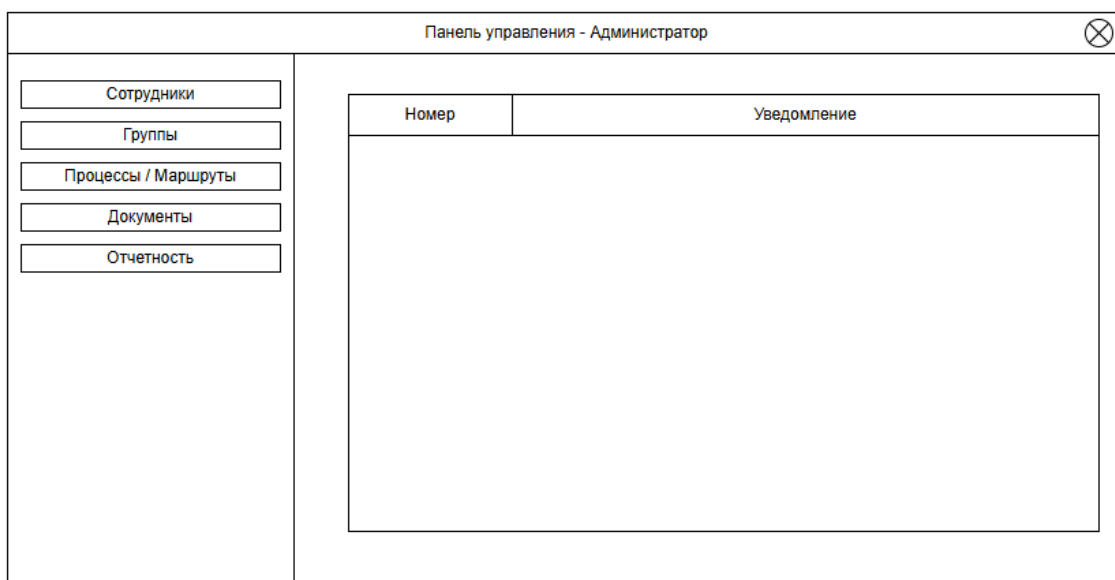


Рисунок 5 - Форма панели управления

Область навигации (слева):

- Кнопка «Сотрудники». При нажатии открывается интерфейс управления сотрудниками. Администратор может просматривать список всех зарегистрированных пользователей, добавлять новых сотрудников, редактировать или удалять существующих. Также реализована возможность назначения ролей (инициатор, согласующий, администратор, наблюдатель).
- Кнопка «Группы». Открывает раздел управления группами пользователей. Администратор может создавать новые группы, добавлять или удалять участников групп, а также назначать группам доступ к определённым маршрутам или документам.
- Кнопка «Процессы / Маршруты». Позволяет администратору просматривать и редактировать маршруты согласования. Реализована возможность создания шаблонов маршрутов, назначения участников процессов на разные этапы и изменения логики согласования (линейное, параллельное и пр.).
- Кнопка «Документы». Переход в раздел со всеми документами, обрабатываемыми в системе. Администратор может просматривать текущий статус документа, вмешиваться в процесс при необходимости (например, изменить маршрут, приостановить или удалить документ).

Центральная область (справа):

- Таблица уведомлений

Отображает актуальные уведомления администратора. Таблица состоит из двух столбцов:

- «Номер» — порядковый номер уведомления;
- «Уведомление» — краткий текст события или действия, требующего внимания (например: «Просрочено согласование документа №42», «Добавлен новый пользователь»).

4.2.4 Функции панели управления сотрудниками

Управление данными о сотрудниках

Номер	ФИО	Должность	Роль
-------	-----	-----------	------

Рисунок 6 — Форма просмотра сотрудников

Центральная таблица сотрудников отображает список всех зарегистрированных сотрудников в системе. Таблица состоит из следующих колонок:

- «Номер» — уникальный идентификатор записи в таблице. Заполняется автоматически при добавлении нового сотрудника.
- «ФИО» — отображает фамилию, имя и отчество сотрудника.
- «Должность» — содержит информацию о должности сотрудника (например: «Менеджер», «Бухгалтер», «Юрист» и т.п.).

При выборе строки возможно редактирование или удаление сотрудника по двойному клику.

4.2.5 Функции панели редактирования данных сотрудника

Панель предоставляет администратору возможность редактировать данные пользователя или удалять его из системы.

Рисунок 7 — Форма редактирования сотрудника

Основная информация формы редактирования:

- Основные данные сотрудника
 - Номер - уникальный ID сотрудника (read-only)
 - ФИО - поле ввода полного имени
 - Должность - поле ввода должности
- Роль - выпадающий список с вариантами:
 - Администратор
 - Согласующий
 - Наблюдатель
- Учётные данные:
 - Логин - уникальный идентификатор для входа
 - Пароль - поле ввода с маскировкой символов (*)

Панель действий (нижняя часть):

- Назад - отмена изменений и возврат
- Удалить - удаление учётной записи (с подтверждением)
- Сохранить - запись изменений в систему

4.2.6 Функционал панели управления группами

Форма позволяет совершать просмотр списка групп и возможность перехода к редактированию

Номер	Название	Состав
-------	----------	--------

Рисунок 8 — Форма просмотра групп

Таблица групп:

- Номер - уникальный идентификатор группы (ID)
- Название - наименование группы (например, "Отдел кадров", "Бухгалтерия")
- Состав - количество сотрудников в группе (возможно с иконкой для просмотра списка)

Элементы управления:

- Добавить - кнопка для создания новой группы
- При двойном нажатии на группу происходит переход к форме редактирования группы

4.2.7 Функционал панели редактирования группы

Панель предоставляет администратору возможность редактировать группы или удалить ее из системы.

Редактирование группы

Сотрудники

Группы

Процессы / Маршруты

Документы

Отчетность

Номер

Название

Состав			
Номер	ФИО	Должность	Роль

Сотрудник

Номер или логин

Добавить

Удалить

Назад

Удалить

Сохранить

Рисунок 9 — Форма редактирования группы

Основные параметры группы (верхняя часть):

- Номер - уникальный идентификатор группы (нередатируемый)
- Название - поле для изменения наименования группы

Таблица текущих участников:

- Номер - ID сотрудника
- ФИО - полное имя участника
- Должность - текущая позиция в организации
- Роль - права в системе (например, "Согласующий")

Панель добавления/удаления:

- Поле "Сотрудник" принимает номер или логин сотрудника
- Кнопка "Добавить" - включает выбранного сотрудника в группу
- Кнопка "Удалить" - убирает выделенного в таблице сотрудника из группы

Панель действий:

- Назад - отмена изменений
- Удалить - полное удаление группы (с подтверждением)
- Сохранить - применение изменений

4.2.8 Функционал панели управления маршрутами

Форма предоставляет возможность просмотра списка маршрутов и возможность перехода к созданию и редактированию маршрутов.

Номер	Название	Путь	Наблюдатели
-------	----------	------	-------------

Рисунок 10 — Форма просмотра маршрутов документов

Таблица маршрутов:

- Номер - уникальный идентификатор маршрута
- Название - произвольное наименование маршрута (например, "Согласование договоров")
- Путь - краткое описание последовательности этапов
- Наблюдатели - количество/список сотрудников с доступом на чтение

Элементы управления:

- Добавить - кнопка для создания нового маршрута согласования
- При двойном нажатии на маршрут в списке происходит переход к панели редактирования

4.2.9 Функционал панели редактирования маршрута

Форма предназначена для редактирования узлов маршрута и списка наблюдателей

Сотрудники

Группы

Процессы / Маршруты

Документы

Отчетность

Редактирование маршрута

Номер

Название

Узлы маршрута

Номер этапа	Номер группы или сотрудника

Этап

Номер группы или сотрудника

Добавить

Удалить

Наблюдатели

Номер	ФИО	Должность	Роль

Сотрудник

Номер или логин

Добавить

Удалить

Назад

Удалить

Сохранить

Рисунок 11 — Форма редактирования маршрута

Основные параметры маршрута:

- Номер - уникальный ID маршрута (нередактируемый)
- Название - поле для изменения наименования маршрута

Управление этапами согласования:

- Таблица текущих этапов:
 - Номер этапа - порядковый номер в маршруте
 - Номер группы или сотрудника - идентификатор ответственного
- Добавление или удаление этапов
 - Кнопка "Добавить" - вставляет новый этап
 - Кнопка "Удалить" - удаляет выбранный этап

Управление наблюдателями

- Таблица наблюдателей:
 - Номер - ID сотрудника

- ФИО - полное имя
- Должность
- Роль в системе
- Панель управления наблюдателями:
 - Поле "Сотрудник": Принимает номер или логин
 - Кнопка "Добавить" - включает сотрудника в наблюдатели
 - Кнопка "Удалить" - исключает выбранного наблюдателя

Панель действий:

- Назад - отмена изменений
- Удалить - полное удаление маршрута (с подтверждением)
- Сохранить - применение изменений

4.2.10 Функциональные требования панели управления документами

Форма для просмотра списка документов и возможность перехода к редактированию.

Номер	Название	Маршрут	Этап	Текущий редактор	Дата создания	Крайний срок
-------	----------	---------	------	------------------	---------------	--------------

Рисунок 12 — Форма просмотра документов

Таблица документов:

- Номер. Уникальный идентификатор документа (ID)
- Название
- Тип документа (маршрут)
- Краткое описание содержимого
- Этап. Текущая стадия маршрута.
- Текущий редактор. Номер сотрудника или группы.
- Дата создания. Дата запуска документа по маршруту.
- Крайний срок. Желаемая дата завершения пути по маршруту.

4.2.11 Функциональные требования панели редактирования состояния документа

Форма предназначена для редактирование параметров и состояния документов.

Редактирование состояния документа

Сотрудники

Группы

Процессы / Маршруты

Документы

Отчетность

Номер

Название

Маршрут

Этап

Редактор

Номер сотрудника или группы

Дата создания

Крайний срок

Версия

Скачать документ

Прикрепить документ

Назад

Удалить

Сохранить

Рисунок 13 — Форма редактирования документа

Информационная панель (верхняя секция)

Параметры документа

- Номер - уникальный идентификатор документа (неизменяемое поле)
- Название - полное наименование документа
- Маршрут - текущий процесс согласования\
- Дата создания - дата и время инициализации
- Версия - текущая редакция документа (формат v.X.Y). Можно выбрать предыдущую и откатиться в прошлую версию.
- Этап. Выпадающий список с доступными этапами
- Редактор. Текущий ответственный (номер сотрудника или группы)
- Крайний срок

Управление файлами:

- Скачать документ. Загрузка документа для локального просмотра и редактирования. Доступные форматы: PDF, DOCX.
- Прикрепить документ. Прикрепить документ. Поддерживаемые форматы: PDF, DOCX.

4.3 Нефункциональные требования

4.3.1 Производительность

Загрузка интерфейса после авторизации пользователя не должна превышать 1 секунды при скорости соединения 1 Мбит/с. Обработка запроса на согласование документа (отправка по маршруту) должна выполняться не более чем за 2 секунды.

4.3.2 Безопасность

- Аутентификация пользователей осуществляется с помощью JWT-токенов.
- Все пароли должны храниться в базе данных в зашифрованном виде с использованием алгоритмов хеширования.
- Все данные между клиентом и сервером передаются по защищённому протоколу HTTPS.

4.3.4 Масштабируемость

- Архитектура приложения должна допускать горизонтальное масштабирование при увеличении числа пользователей.
- Возможность раздельного масштабирования backend и frontend компонентов.

4.3.4 Технические требования

4.3.4.1 Клиентская часть

Используемые технологии:

- Java (Spring Boot + Spring MVC) — для реализации веб-интерфейса и обработки HTTP-запросов.
- JavaFX — при необходимости разработки клиента с графическим интерфейсом[20].

Spring MVC позволяет эффективно организовать обработку клиентских запросов и маршрутизацию, а JavaFX используется для локального взаимодействия с файлами и сетевыми ресурсами, если это необходимо. Все технологии обеспечивают

кроссплатформенность и хорошую интеграцию с Java [19].

4.3.4.2 Серверная часть

Используемые технологии:

- Java 17+ — актуальная LTS-версия языка программирования с поддержкой современных возможностей и долгосрочной поддержкой.
- Spring Boot — упрощает создание REST API и управление зависимостями.
- Spring Security — реализует аутентификацию и авторизацию пользователей.
- Maven — используется для управления зависимостями и сборкой проекта.
- Docker — контейнеризация серверного приложения и базы данных для унификации среды исполнения и упрощения развёртывания.
- Samba — подключение и работа с сетевыми файловыми хранилищами (например, доступ к документам, расположенным на файловом сервере в локальной сети).

Spring Boot ускоряет создание серверной части за счёт автоматической конфигурации и модульности. Spring Security предоставляет гибкие механизмы контроля доступа [16, 17]. Docker позволяет запускать приложение и сопутствующие сервисы (БД, Samba) в изолированных и воспроизводимых контейнерах, снижая зависимость от настроек среды. Использование Samba необходимо для интеграции с корпоративной инфраструктурой хранения документов, поддерживающей общий доступ по SMB-протоколу.

4.3.4.3 База данных

Используемые технологии:

- PostgreSQL 15 или выше — мощная реляционная СУБД с поддержкой JSONB, транзакций и индексов.
- Spring Data JPA + Hibernate — ORM-слой для работы с базой данных через объектную модель.
- Docker — для контейнеризации PostgreSQL в рамках общей среды.

PostgreSQL обеспечивает надёжность, масштабируемость и расширенные функции работы с данными. ORM-инструменты упрощают реализацию слоя доступа к данным и повышают читаемость кода.

5 Разработка и тестирование

5.1 Среда разработки

Программа разрабатывалась на языке Java с использованием среды IntelliJ IDEA — это удобный инструмент для редактирования кода и управления проектом. Для создания графического интерфейса применялся SceneBuilder, который позволяет быстро собирать окна и формы. Все компоненты системы размещены в Docker-контейнерах — это упрощает запуск приложения на разных компьютерах, а также облегчает его настройку и обновление. Такой подход обеспечивает удобство работы и стабильность программы

5.2 Схема проекта

Сервер взаимодействует с двумя основными внешними системами:

- База данных Postgres — сюда сервер отправляет и откуда получает структурированные данные, такие как информация о пользователях, маршрутах согласования, статусах документов и т. д.
- SMB-сервер (Samba) — используется для хранения файлов документов. Сервер отправляет и получает файлы через этот файловый сервер, обеспечивая централизованное и защищённое хранение документов.

Таким образом, клиенты напрямую не работают ни с базой данных, ни с файловым сервером. Все операции проходят через сервер (backend), что обеспечивает безопасность, централизованный контроль доступа и целостность данных.

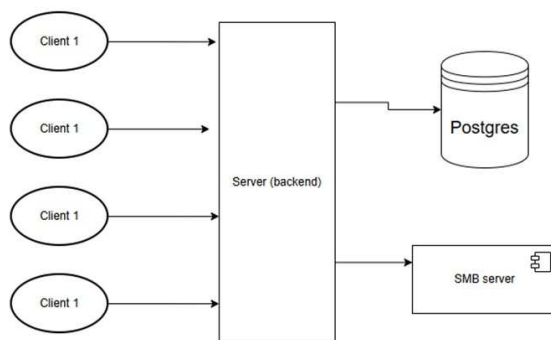


Рисунок 14 — Схема проекта

5.3 База данных

Для хранения данных сущностей в проекте используется реляционная база данных PostgreSQL, структура которой спроектирована с учетом основных требований системы

согласования документов. Модель данных включает несколько ключевых таблиц, связанных между собой через внешние ключи, что обеспечивает целостность и удобство работы с информацией.

Основная таблица `user` содержит данные о пользователях системы: уникальный идентификатор, имя пользователя, зашифрованный пароль, полное имя, электронную почту, должность и подразделение. Для группировки пользователей предусмотрена таблица `usergroup`, где хранятся сведения о группах, включая их идентификаторы, названия и описания. Связь многие-ко-многим между пользователями и группами реализована через промежуточную таблицу `user_user_groups`.

Таблица `executor` отвечает за исполнителей в системе и может ссылаться либо на отдельного пользователя, либо на группу пользователей, что позволяет гибко настраивать маршруты согласования. Таблицы `route` и `route_node` описывают маршруты согласования и их этапы, где каждый этап связан с конкретным исполнителем и имеет порядок выполнения.

Документы хранятся в таблице `document`, которая содержит информацию о названии, описании, статусе, текущем этапе согласования, инициаторе, маршруте и других параметрах. Для управления версиями документов предусмотрена таблица `version`, где фиксируются пути к файлам и контрольные суммы, обеспечивая возможность отслеживания изменений и восстановления предыдущих версий.

Все связи между таблицами реализованы через внешние ключи, что гарантирует согласованность данных и упрощает выполнение сложных запросов. Такая структура базы данных позволяет эффективно хранить и обрабатывать информацию, необходимую для работы системы согласования документов, обеспечивая при этом безопасность и масштабируемость.

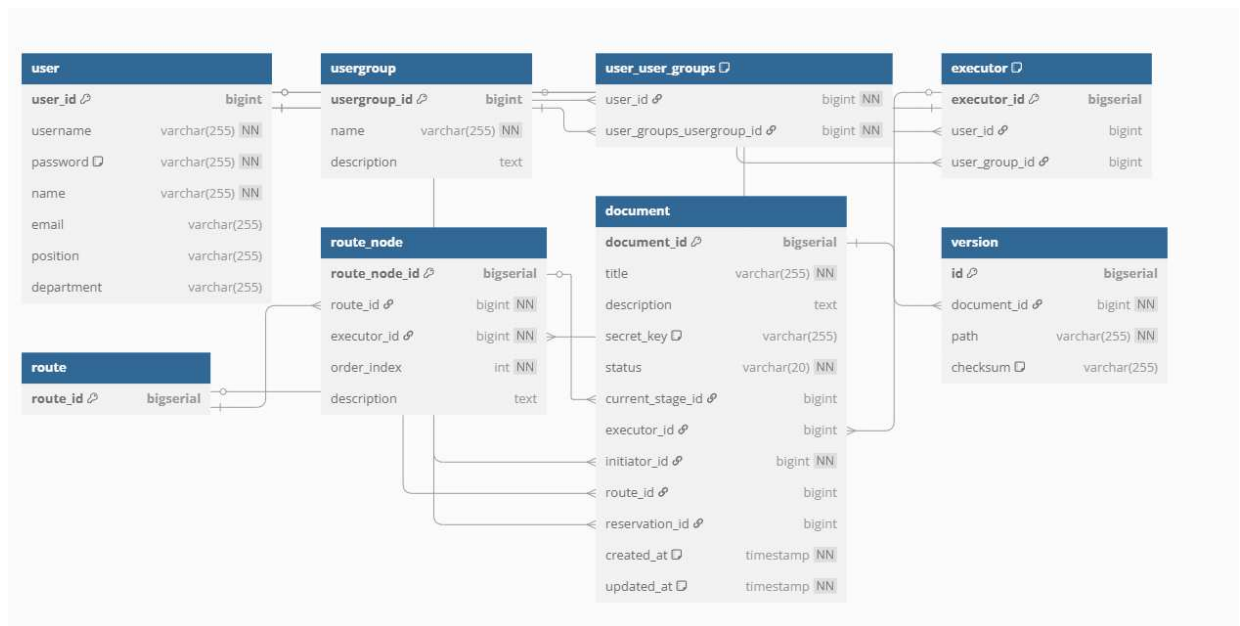


Рисунок 15 — Модель базы данных

5.4 Структура серверной части

Компоненты системы структурированы по пакетам

- controller — обработка HTTP-запросов, связь клиента с сервером
- entity — классы, отображающие структуру данных в БД (пользователи, документы, маршруты)
- repository — интерфейсы для работы с базой данных
- service — реализация бизнес-логики
- smb — работа с файлами на Samba-сервере
- utils — вспомогательные классы

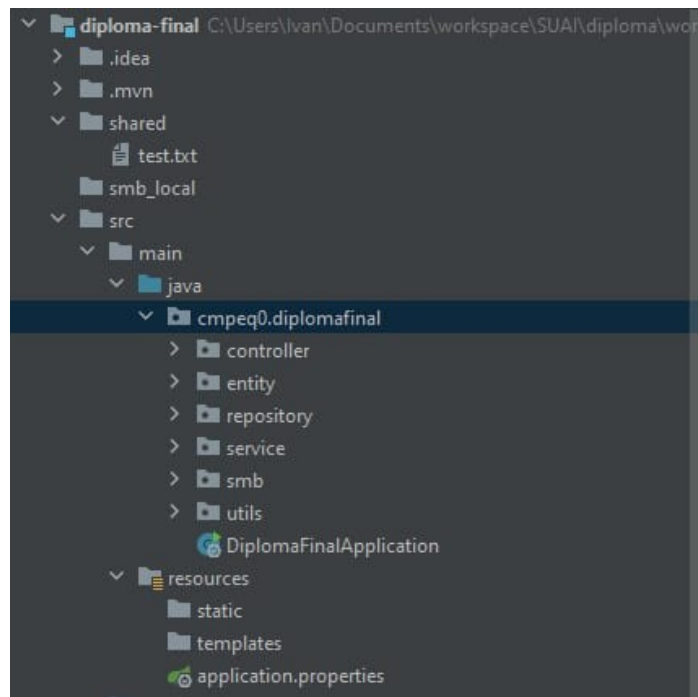


Рисунок 16 — Структура серверной части

5.5 Реализованные контроллеры

Данные контроллеры отвечают за входящие на REST API запросы и реализуют следующие функции системы:

- AuthController — вход и права пользователей
- UserController — управление пользователями
- GroupController — работа с группами
- DocumentController — документы
- ApprovalRouteController — согласование по маршрутам
- NotificationController — уведомления
- VersionController — версии документов

5.6 Используемые технологии

В ходе работы было использовано следующее сочетание технологий, объединенных через систему сборки Maven.

- Maven — сборка проекта и подключение библиотек

- Spring Boot Data JPA — работа с базой данных PostgreSQL
- Spring Boot Web — реализация REST API
- Spring Boot Docker Compose — запуск зависимостей через Docker
- PostgreSQL JDBC Driver — подключение к PostgreSQL
- Lombok — упрощение кода за счет аннотаций
- Spring Integration SMB — работа с файлами на Samba

5.7 Шифрование файлов

Файлы документов хранятся на SMB-сервере в зашифрованном виде с помощью алгоритма AES, что обеспечивает высокий уровень конфиденциальности информации. Ключ для расшифровки файлов хранится в базе данных PostgreSQL и используется только при авторизованном доступе к файлу. Доступ к SMB-серверу и расшифровка файлов реализованы в виде отдельных компонент JWTService и SMBService.

AES — это современный симметричный алгоритм шифрования, который использует один и тот же ключ для зашифровки и расшифровки данных. Он работает с блоками информации по 128 бит, а длина ключа может составлять 128, 192 или 256 бит, что обеспечивает высокий уровень защиты. Благодаря своей скорости и надёжности AES широко применяется для защиты конфиденциальных данных в самых разных областях.

Алгоритм AES-256 разрешён российским законодательством, в том числе ФЗ №63 "Об электронной подписи", а также приказами ФСБ России №378 и №239, при условии, что ключи создаются надёжными методами и имеют длину не менее 256 бит. Таким образом, AES-256 может применяться для защиты конфиденциальных данных и электронных подписей в коммерческих и частных системах.

5.8 Развертывание

Для развертывания системы применяется Docker Compose, включающий следующие сервисы: PostgreSQL для хранения информации о пользователях, документах и маршрутах; Samba в качестве файлового сервера для документов и их версий; и основное backend-приложение (API) под названием App. Для работы системы требуется установленные Docker и Docker Compose, а также открытые порты 5432 (для PostgreSQL) и 8082 (для приложения)

Для запуска сервисов достаточно одной команды «docker-compose up -d». Сервисы будут развернуты в фоновом режиме.

5.9 Клиентская часть

Клиентская часть системы реализована в виде настольного приложения, построенного на базе Spring Framework и JavaFX. Такой подход позволяет совместить мощь Spring для организации логики и интеграции с внешними сервисами, а также преимущества JavaFX для создания современного и отзывчивого графического интерфейса. Вся логика взаимодействия пользователя с приложением, включая формы ввода, таблицы, кнопки и навигацию, реализована с помощью JavaFX-компонентов, что обеспечивает удобство работы и привычный внешний вид для пользователей настольных систем.

Для получения данных с сервера используется REST API, предоставляемый backend-частью системы. Клиентское приложение отправляет HTTP-запросы к серверу через стандартные механизмы Spring Rest Template, получает ответы в формате JSON и отображает их в графическом интерфейсе. Такой подход обеспечивает гибкость, масштабируемость и простоту интеграции — клиент работает независимо от сервера, а все изменения данных синхронизируются через API.

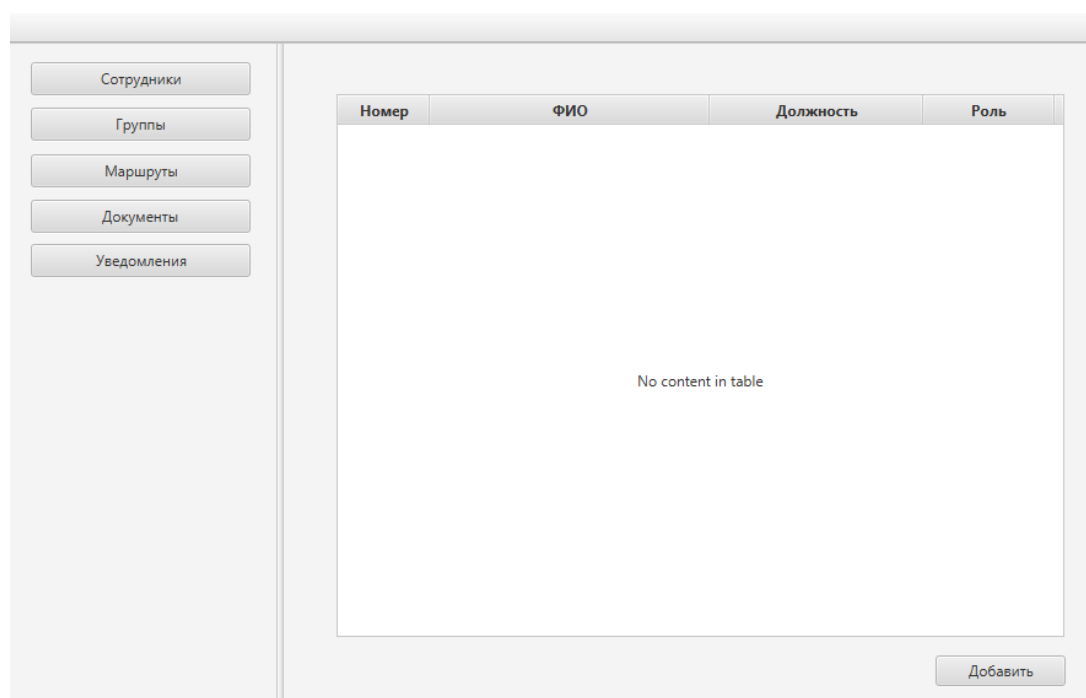


Рисунок 17 - Реализованное окно просмотра сотрудников

5.10 Тестирование

5.10.1 Ключевые аспекты

В ходе тестирования выполняются задачи по ручной проверке всех ключевых функций системы, оценке её поведения в нестандартных сценариях и фиксации выявленных ошибок и рекомендаций. Также проводится тестирование взаимодействия между модулями, а по итогам формируются объективные выводы о качестве и готовности продукта к внедрению в эксплуатацию.

5.10.2 Проверяемые сценарии

Сценарий	Ожидаемый результат
Регистрация нового пользователя. Администратор вводит корректные данные (логин, пароль) и нажимает кнопку регистрации.	Пользователь успешно зарегистрирован, появляется возможность входа в систему.
Вход в систему с корректными данными. Пользователь вводит правильные login и пароль, нажимает кнопку входа.	Пользователь авторизуется, получает доступ к основному функционалу системы.
Вход с неверными учетными данными. Пользователь вводит неверный логин или пароль.	Система выводит сообщение об ошибке, вход не выполняется.
Создание нового документа. Пользователь заполняет форму создания документа (название, описание, загружает файл), нажимает кнопку "Создать".	Документ создан, появляется в списке документов, файл загружен на сервер.
Пользователь открывает раздел с документами.	Отображается список всех доступных пользователю документов с краткой информацией о каждом.
Пользователь создаёт новый маршрут, добавляет этапы и назначает согласующих.	Маршрут успешно создан, этапы и согласующие отображаются в интерфейсе.
Согласующий выбирает документ и изменяет его этап.	Проверяется возможность перехода на выбранный этап. Статус документа обновляется, инициатор и другие согласующие получают уведомления.
Загрузка документа с недопустимым форматом (например, .exe).	Система отклоняет файл, выводит сообщение о поддерживаемых форматах.
Добавление этапа маршрута без указания исполнителя.	Система отклоняет изменение, выводит сообщение о необходимости завершить маршрут.
Добавление пользователя в группу, к которой он уже принадлежит.	Система игнорирует дублирование, выводит предупреждение.
Изменение роли пользователя с "USER" на "ADMIN".	Роль успешно изменена, пользователь получает новые права.

Попытка создания маршрута без этапов.	Система отклоняет запрос, выводит сообщение о необходимости добавить этапы.
Попытка доступа к документу без авторизации	Система отклоняет запрос.
Попытка доступа к API с некорректным JWT-токеном.	Система возвращает ошибку 401 (Unauthorized).
Проверка работы уведомлений при смене этапа согласования.	Уведомления отправляются всем участникам нового этапа.

5.10.3 Итоги тестирования

По итогам тестирования все ключевые сценарии были успешно выполнены — система корректно обрабатывает действия пользователей, обеспечивает стабильную работу основных и дополнительных функций, а также корректно отображает сообщения и уведомления в пользовательском интерфейсе (UI). Это подтверждает, что система готова к использованию и обеспечивает необходимый уровень удобства и надёжности.

Заключение

В ходе дипломного проекта была создана специализированная система, предназначенная для автоматизации и поддержки процессов согласования документов. В основу разработки легли современные технологии и актуальные принципы проектирования программных систем. На подготовительном этапе проведён анализ существующих решений по организации документооборота, сформулированы основные требования к будущей системе, а также выполнено проектирование и последующая реализация рабочего прототипа.

Программный комплекс позволяет автоматизировать выполнение стандартных задач, связанных с согласованием документов, что положительно сказывается на прозрачности и контроле текущих процессов. Использование системы способствует сокращению времени на согласование и снижению трудозатрат сотрудников, вовлечённых в документооборот. Проведённые испытания подтвердили работоспособность всех ключевых функций, удобство интерфейса и возможность эффективного применения системы в условиях реальной организации.

Данная разработка открывает перспективы для дальнейшего развития и адаптации программного продукта под нужды различных предприятий и учреждений. Полученные результаты могут послужить основой для дальнейших исследований и внедрения цифровых технологий в процессы управления документами.

Список использованных источников

1. Баранов В.А., Кузнецов А.В. Электронный документооборот: теория и практика. — М.: Горячая линия – Телеком, 2017. — 304 с.
2. Костров А.В. Системы электронного документооборота: учебное пособие. — СПб.: Питер, 2020. — 288 с.
3. Кузнецов С.В., Михайлов А.В. Автоматизация документооборота на предприятии. — М.: Инфра-М, 2018. — 256 с.
4. Михайлов В.Г., Лапшин А.А. Информационные системы управления документооборотом. — М.: БХВ-Петербург, 2019. — 320 с.
5. Глушков В.М. Основы безбумажной информационной технологии. — М.: Наука, 2016. — 272 с.
6. ГОСТ Р 6.30-2003. Унифицированные системы документации. Требования к оформлению документов.
7. ГОСТ Р 7.0.97–2016. СИБИД. Электронные документы. Термины и определения.
8. ФЗ-152 "О персональных данных" (с изменениями и дополнениями) // КонсультантПлюс URL: http://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 26.05.2025).
9. ISO 15489-1:2016 Information and documentation — Records management — Part 1: Concepts and principles // <https://www.iso.org/standard/62542.html> (дата обращения: 26.05.2025).
10. Smith R. Document Management for the Enterprise. — Wiley, 2021. — 412 p.
11. Brown A., Grant G. Document Automation and Workflow Management. — Springer, 2019. — 368 p.
12. DMS Alfresco: Architecture and Functionality. — Packt Publishing, 2020. — 254 p.
13. Electronic Document Management Systems: A Review // Journal of Information Systems, 2023, Vol. 37, No. 2, pp. 120–134.
14. Кузнецов А.В. Документооборот в современных организациях // Информационные технологии. — 2022. — №2. — С. 34–41.
15. Иванова О.Н. Автоматизация бизнес-процессов согласования документов // Вестник ИТ. — 2021. — №4. — С. 55–62.

16. Кинг Дж., Хеллер Г. Spring 5 для профессионалов. — М.: Диалектика, 2020. — 800 с.
17. Уоллс К. Spring в действии. 6-е издание. — СПб.: Питер, 2022. — 624 с.
18. Гудхейрт Х. JavaFX для профессионалов. — М.: ДМК Пресс, 2021. — 480 с.
19. Фримен К., Фримен Э. Spring. Руководство для начинающих. — СПб.: Питер, 2021. — 400 с.
20. Дей Х. JavaFX. Полное руководство. — М.: Вильямс, 2020. — 688 с.

Приложение А. Скрипт формирования базы данных.

-- Создание таблицы пользователей

```
CREATE TABLE "user" (  
    user_id BIGINT PRIMARY KEY,  
    username VARCHAR(255) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL, -- Хэшированный пароль  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255),  
    position VARCHAR(255),  
    department VARCHAR(255)  
);
```

-- Создание таблицы групп пользователей

```
CREATE TABLE usergroup (  
    usergroup_id BIGINT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    description TEXT  
);
```

-- Таблица связи многие-ко-многим между пользователями и группами

```
CREATE TABLE user_user_groups (  
    user_id BIGINT NOT NULL,  
    user_groups_usergroup_id BIGINT NOT NULL,  
    PRIMARY KEY (user_id, user_groups_usergroup_id),  
    FOREIGN KEY (user_id) REFERENCES "user" (user_id),  
    FOREIGN KEY (user_groups_usergroup_id) REFERENCES usergroup (usergroup_id)
```

```
);
```

```
-- Создание таблицы исполнителей
```

```
CREATE TABLE executor (
```

```
    executor_id BIGSERIAL PRIMARY KEY,
```

```
    user_id BIGINT,
```

```
    user_group_id BIGINT,
```

```
    FOREIGN KEY (user_id) REFERENCES "user" (user_id),
```

```
    FOREIGN KEY (user_group_id) REFERENCES usergroup (usergroup_id),
```

```
    CONSTRAINT chk_executor CHECK (user_id IS NOT NULL OR user_group_id IS NOT NULL)
```

```
);
```

```
-- Создание таблицы маршрутов
```

```
CREATE TABLE route (
```

```
    route_id BIGSERIAL PRIMARY KEY
```

```
);
```

```
-- Создание таблицы этапов маршрута
```

```
CREATE TABLE route_node (
```

```
    route_node_id BIGSERIAL PRIMARY KEY,
```

```
    route_id BIGINT NOT NULL,
```

```
    executor_id BIGINT NOT NULL,
```

```
    order_index INT NOT NULL,
```

```
    description TEXT,
```

```
    FOREIGN KEY (route_id) REFERENCES route (route_id),
```

```

FOREIGN KEY (executor_id) REFERENCES executor (executor_id)

);

-- Создание таблицы документов

CREATE TABLE document (

    document_id BIGSERIAL PRIMARY KEY,

    title VARCHAR(255) NOT NULL,

    description TEXT,

    secret_key VARCHAR(255), -- Секретный ключ для документа

    status VARCHAR(20) NOT NULL,

    current_stage_id BIGINT,

    executor_id BIGINT,

    initiator_id BIGINT NOT NULL,

    route_id BIGINT,

    reservation_id BIGINT,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (current_stage_id) REFERENCES route_node (route_node_id),

    FOREIGN KEY (executor_id) REFERENCES executor (executor_id),

    FOREIGN KEY (initiator_id) REFERENCES "user" (user_id),

    FOREIGN KEY (route_id) REFERENCES route (route_id),

    FOREIGN KEY (reservation_id) REFERENCES "user" (user_id)

);

-- Создание таблицы версий документов

CREATE TABLE version (

```

```
id BIGSERIAL PRIMARY KEY,  
document_id BIGINT NOT NULL,  
path VARCHAR(255) NOT NULL,  
checksum VARCHAR(255), -- Контрольная сумма файла  
FOREIGN KEY (document_id) REFERENCES document (document_id)  
);
```

-- Создание индексов для улучшения производительности

```
CREATE INDEX idx_document_status ON document (status);  
CREATE INDEX idx_document_initiator ON document (initiator_id);  
CREATE INDEX idx_route_node_route ON route_node (route_id);  
CREATE INDEX idx_version_document ON version (document_id);  
CREATE INDEX idx_user_username ON "user" (username);  
CREATE INDEX idx_document_secret_key ON document (secret_key);
```

Приложение Б. Получение и шифрование файлов

FileEncryptionService.java

```
package cmpeq0.diplomafinal.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import javax.crypto.*;
import javax.crypto.spec.IvParameterSpec;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.security.spec.InvalidKeySpecException;

@Service

public class FileEncryptionService {

    private static final String ALGORITHM = "AES/CBC/PKCS5Padding";

    @Autowired

    private FileKeyService fileKeyService;

    public void encryptFile(Path inputFile, Path outputFile, String filename, String password)

        throws NoSuchPaddingException, NoSuchAlgorithmException, InvalidKeyException,
```

```
InvalidAlgorithmParameterException, IllegalBlockSizeException,  
BadPaddingException,  
IOException, InvalidKeySpecException {
```

```
SecretKey secretKey = fileKeyService.getSecretKeyForFile(filename, password);  
byte[] iv = fileKeyService.getIvForFile(filename);
```

```
Cipher cipher = Cipher.getInstance(ALGORITHM);  
cipher.init(Cipher.ENCRYPT_MODE, secretKey, new IvParameterSpec(iv));
```

```
byte[] inputBytes = Files.readAllBytes(inputFile);  
byte[] encryptedBytes = cipher.doFinal(inputBytes);
```

```
Files.write(outputFile, encryptedBytes);
```

```
}
```

```
public void decryptFile(Path inputFile, Path outputFile, String filename, String password)  
    throws NoSuchPaddingException, NoSuchAlgorithmException, InvalidKeyException,
```

```
InvalidAlgorithmParameterException, IllegalBlockSizeException,  
BadPaddingException,  
IOException, InvalidKeySpecException {
```

```
SecretKey secretKey = fileKeyService.getSecretKeyForFile(filename, password);  
byte[] iv = fileKeyService.getIvForFile(filename);
```

```
Cipher cipher = Cipher.getInstance(ALGORITHM);  
cipher.init(Cipher.DECRYPT_MODE, secretKey, new IvParameterSpec(iv));
```

```

        byte[] encryptedBytes = Files.readAllBytes(inputFile);

        byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

        Files.write(outputFile, decryptedBytes);
    }
}

```

SmbConfig.java

```

package cmpeq0.diplomafinal.smb;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.smb.session.SmbSessionFactory;

@Configuration
public class SmbConfig {

    @Value("${smb.host}")
    private String smbHost;

    @Value("${smb.port}")
    private int smbPort;

    @Value("${smb.domain}")

```

```
private String smbDomain;
```

```
@Value("${smb.username}")
```

```
private String smbUsername;
```

```
@Value("${smb.password}")
```

```
private String smbPassword;
```

```
@Value("${smb.share}")
```

```
private String smbShare;
```

```
@Value("${smb.base-path}")
```

```
private String smbBasePath;
```

```
@Bean
```

```
public SmbSessionFactory smbSessionFactory() {
```

```
    SmbSessionFactory factory = new SmbSessionFactory();
```

```
    factory.setHost(smbHost);
```

```
    factory.setPort(smbPort);
```

```
    factory.setDomain(smbDomain);
```

```
    factory.setUsername(smbUsername);
```

```
    factory.setPassword(smbPassword);
```

```
    factory.setShareAndDir(smbShare + smbBasePath);
```

```
    return factory;
```

```
}
```

```
}
```

SmbService.java

```
package cmpeq0.diplomafinal.smb;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.integration.smb.session.SmbSession;
```

```
import org.springframework.integration.smb.session.SmbSessionFactory;
```

```
import org.springframework.stereotype.Service;
```

```
import java.io.*;
```

```
import java.nio.charset.StandardCharsets;
```

```
import java.nio.file.Files;
```

```
import java.nio.file.Path;
```

```
import java.nio.file.Paths;
```

```
import java.nio.file.StandardCopyOption;
```

```
@Service
```

```
public class SmbService {
```

```
    private final SmbSessionFactory smbSessionFactory;
```

```
    private final Path localCacheDir;
```

```
    public SmbService(SmbSessionFactory smbSessionFactory, @Value("${local.storage}")  
String cachePath) throws IOException{
```

```
        this.smbSessionFactory = smbSessionFactory;
```

```

this.localCacheDir = Paths.get(cachePath).toAbsolutePath().normalize();

if (!Files.exists(localCacheDir)) {
    Files.createDirectories(localCacheDir);
}
}

public File getFile(String smbPath) throws IOException {
    Path localPath = getLocalCachePath(smbPath);

    if (isFileCached(localPath)) {
        return localPath.toFile();
    }

    // Загружаем с SMB сервера
    try (SmbSession session = smbSessionFactory.getSession()) {
        if (!session.exists(smbPath)) {
            throw new FileNotFoundException("File not found on SMB server: " + smbPath);
        }

        // Скачиваем файл во временный файл
        Path tempFile = Files.createTempFile(localCacheDir, "download_", ".tmp");

        try (InputStream smbStream = session.readRaw(smbPath);
            OutputStream fileStream = new FileOutputStream(tempFile.toFile())) {
            byte[] buffer = new byte[4096];
            int bytesRead;

```

```

        while ((bytesRead = smbStream.read(buffer)) != -1) {
            fileStream.write(buffer, 0, bytesRead);
        }
    }

    // Перемещаем временный файл в конечное положение
    Files.move(tempFile, localPath, StandardCopyOption.REPLACE_EXISTING);

    return localPath.toFile();
}

}

public void uploadFile(String smbPath, File localFile) throws IOException {
    Path cachedPath = getLocalCachePath(smbPath);
    Files.copy(localFile.toPath(), cachedPath, StandardCopyOption.REPLACE_EXISTING);

    try (SmbSession session = smbSessionFactory.getSession();
        InputStream fileStream = new FileInputStream(localFile)) {
        session.write(fileStream, smbPath);
    }
}

public String getFileContent(String fileName) throws IOException {
    try (SmbSession session = smbSessionFactory.getSession()){
        InputStream inputStream = session.readRaw(fileName);
    }
}

```

```

        ByteArrayOutputStream data = new ByteArrayOutputStream();

        byte[] buffer = new byte[4096];

        int length;

        while ((length = inputStream.read(buffer)) != -1) {

            data.write(buffer, 0, length);

        }

        return data.toString(StandardCharsets.UTF_8);

    }

}

public boolean fileExists(String filePath) throws IOException {

    try (SmbSession session = smbSessionFactory.getSession()) {

        return session.exists(filePath);

    }

}

private boolean isFileCached(Path localPath) throws IOException {

    return Files.exists(localPath);

}

private Path getLocalCachePath(String smbPath) {

    // Заменяем недопустимые символы в имени файла

    String safeFileName = smbPath.replaceAll("[\\\\\\/:*?\"<>|]", "_");

    return localCacheDir.resolve(safeFileName);

}

}

```

Приложение В. Аутентификация с JWT

TokenInterceptor.java

```
package cmpeq0.diplomafinal.jwt;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@RequiredArgsConstructor
public class TokenInterceptor extends OncePerRequestFilter {

    private final TokenEngine tokenEngine;

    private final CustomUserDetailsService userService;

    @Override
```

```

protected void doFilterInternal(HttpServletRequest req,
                                HttpServletResponse res,
                                FilterChain chain) throws ServletException, IOException {
    try {
        String token = extractToken(req);

        if (StringUtils.hasText(token) && tokenEngine.isTokenValid(token)) {
            String login = tokenEngine.extractLogin(token);

            UserDetails user = userService.loadUserByUsername(login);

            UsernamePasswordAuthenticationToken auth = new
UsernamePasswordAuthenticationToken(
                user, null, user.getAuthorities());

            auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(req));

            SecurityContextHolder.getContext().setAuthentication(auth);
        }
    } catch (Exception ex) {
        logger.error("Authentication setup failed", ex);
    }

    chain.doFilter(req, res);
}

private String extractToken(HttpServletRequest req) {
    String header = req.getHeader("Authorization");

```

```

        return (StringUtils.hasText(header) && header.startsWith("Bearer ")) ? header.substring(7) :
null;

    }

}

```

TokenEngine.java

```

package cmpeq0.diplomafinal.jwt;

import io.jsonwebtoken.*;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;

@Slf4j
@Component
public class TokenEngine {

    @Value("${jwt.secret}")
    private String keySecret;

    @Value("${jwt.expiration-ms}")

```

```
private int tokenLifespanMs;
```

```
public String createToken(Authentication auth) {
```

```
    Date now = new Date();
```

```
    Date expiry = new Date(now.getTime() + tokenLifespanMs);
```

```
    return Jwts.builder()
```

```
        .setSubject(auth.getName())
```

```
        .setIssuedAt(now)
```

```
        .setExpiration(expiry)
```

```
        .signWith(getSigningKey(), SignatureAlgorithm.HS256)
```

```
        .compact();
```

```
}
```

```
public String extractLogin(String token) {
```

```
    return Jwts.parserBuilder()
```

```
        .setSigningKey(getSigningKey())
```

```
        .build()
```

```
        .parseClaimsJws(token)
```

```
        .getBody()
```

```
        .getSubject();
```

```
}
```

```
public boolean isTokenValid(String token) {
```

```
    try {
```

```
        Jwts.parserBuilder().setSigningKey(getSigningKey()).build().parseClaimsJws(token);
```

```
        return true;
```

```

    } catch (JwtException | IllegalArgumentException e) {

        log.warn("Token validation failed: {}", e.getMessage());

    }

    return false;

}

private Key getSigningKey() {

    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(keySecret));

}

}

```

SecurityConfig.java

```

package cmpeq0.diplomafinal.jwt;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConf
iguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

```

@Configuration

@EnableWebSecurity

public class SecurityConfiguration {

private final JwtAuthEntryPoint entryPoint;

public SecurityConfiguration(JwtAuthEntryPoint entryPoint) {

 this.entryPoint = entryPoint;

}

@Bean

public SecurityFilterChain securityRules(HttpSecurity http) throws Exception {

 http

 .csrf().disable()

 .exceptionHandling().authenticationEntryPoint(entryPoint).and()

 .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS).and()

 .authorizeRequests()

 .antMatchers("/api/auth/**", "/h2-console/**").permitAll()

 .anyRequest().authenticated().and()

 .headers().frameOptions().sameOrigin().and()

 .httpBasic();

 http.addFilterBefore(authTokenInterceptor(), UsernamePasswordAuthenticationFilter.class);

 return http.build();

}

@Bean

```
public AuthenticationManager authManager(AuthenticationConfiguration config) throws  
Exception {  
    return config.getAuthenticationManager();  
}
```

@Bean

```
public PasswordEncoder hashEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

@Bean

```
public TokenInterceptor authTokenInterceptor() {  
    return new TokenInterceptor(tokenEngine(), userDetailsService());  
}
```

@Bean

```
public TokenEngine tokenEngine() {  
    return new TokenEngine();  
}
```

@Bean

```
public CustomUserDetailsService userDetailsService() {  
    return new CustomUserDetailsService();  
}  
}
```

Приложение Г. Основные сущности

Document.java

```
package cmpeq0.diplomafinal.entity;

import jakarta.persistence.*;
import lombok.Data;

@Data
@Entity
@Table(name = "document")
public class Document {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "document_id")
    private long id;

    private String title;

    private String description;

    @ManyToOne
    private RouteNode currentStage;

    @ManyToOne
    private Executor executor;
```

```
@ManyToOne  
  
private User reservation;  
  
}
```

Executor.java

```
package cmpeq0.diplomafinal.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```
@Data
```

```
@Entity
```

```
@Table(name = "executor")
```

```
public class Executor {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "executor_id")
```

```
    private long id;
```

```
    @OneToOne
```

```
    private User user;
```

```
    @OneToOne
```

```
    private UserGroup userGroup;
```

```
}
```

Route.java

```
package cmpeq0.diplomafinal.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```
import java.util.List;
```

```
@Data
```

```
@Entity
```

```
@Table(name = "route")
```

```
public class Route {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "route_id")
```

```
    private long id;
```

```
    @OneToMany(mappedBy = "route", cascade = CascadeType.ALL, orphanRemoval = true)
```

```
    @OrderBy("orderIndex ASC")
```

```
    private List<RouteNode> stages;
```

```
}
```

RouteNode.java

```
package cmpeq0.diplomafinal.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```
@Data
```

```
@Entity
```

```
@Table(name = "route_node")
```

```
public class RouteNode {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "route_node_id")
```

```
    private long id;
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "route_id")
```

```
    private Route route;
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "executor_id")
```

```
    private Executor target;
```

```
    private int orderIndex;
```

```
    private String description;  
}
```

User.java

```
package cmpeq0.diplomafinal.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```
import java.util.List;
```

```
@Data
```

```
@Entity
```

```
@Table(name = "user")
```

```
public class User {
```

```
    @Id
```

```
    @Column(name = "user_id")
```

```
    private long id;
```

```
    private String name;
```

```
    @ManyToMany
```

```
    private List<UserGroup> userGroups;
```

```
}
```

UserGroup.java

```
package cmpeq0.diplomafinal.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```
import java.util.List;
```

```
@Data
```

```
@Entity
```

```
@Table(name = "usergroup")
```

```
public class UserGroup {
```

```
    @Id
```

```
    @Column(name = "usergroup_id")
```

```
    private long id;
```

```
    private String name;
```

```
    private String description;
```

```
    @ManyToMany
```

```
    private List<User> users;
```

```
}
```

Приложение Д. Конфигурация docker compose.yaml

version: '3.8'

services:

postgres:

image: postgres

environment:

POSTGRES_DB: diploma

POSTGRES_USER: user

POSTGRES_PASSWORD: password

volumes:

- pg_data:/var/lib/postgresql/data

ports:

- "5432:5432"

networks:

- backend

samba:

image: dperson/samba

command: -s "shared;/mount;yes;no;no;user" -u "user;password"

environment:

USERID: 1000

GROUPID: 1000

volumes:

- ./shared:/mount

networks:

- backend

restart: always

app:

build: .

depends_on:

postgres:

- condition: service_healthy

samba:

- condition: service_started

networks:

- backend

ports:

- "8082:8082"

environment:

SAMBA_SHARE_PATH: //samba/shared

SAMBA_USER: user

SAMBA_PASS: password

volumes:

pg_data:

networks:

backend:

driver: bridge

Dockerfile

FROM eclipse-temurin:22-jdk

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8082

ENTRYPOINT ["java", "-jar", "app.jar"]

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ОТЗЫВ РУКОВОДИТЕЛЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

на тему Разработка программного комплекса для автоматизации процессов
согласования документов

выполненную студентом группы № 4132

Гусевым Иваном Евгеньевичем

фамилия, имя, отчество студента

по направлению подготовки/ 09.03.04
специальности (код)

Программная инженерия

(наименование направления подготовки/специальности)

Проектирование программных систем

(наименование направления подготовки/специальности)

Актуальность темы работы:

Процессы согласования изменений в документации, терминах и определениях так или иначе возникают в любой крупной организации. Гусев И.Е. рассмотрел вопросы автоматизации процессов согласования документов.

Цель и задачи работы:

Цель дипломной работы — создание программного комплекса, обеспечивающего автоматизацию процессов согласования документов в организациях различных масштабов.

Общая оценка выполнения поставленной перед студентом задачи, основные достоинства и недостатки работы:

В ходе выполнения ВКР Гусев И.Е. описал предметную область, выполнил сравнение с аналогами, разработал серверную часть программного комплекса автоматизации процессов согласования документов, разработал макеты форм пользовательского интерфейса. При создании серверной части системы согласования документов Гусев И.Е. использовал язык программирования Java, фреймворк Spring Boot, систему управления базами данных PostgreSQL, пакет программ доступа к файлам Samba.

Степень самостоятельности и способности к исследовательской работе студента:

В ходе работы Гусев И.Е. проявил способности к исследовательской работе, умение работы с различными источниками информации, навыки поиска и обобщения материала, необходимого для реализации системы, формулирования выводов.

Проверка текста выпускной квалификационной работы с использованием системы antiplagiat, проводившаяся 15.06.2025, показывает оригинальность содержания на уровне 95.97%. Текст пояснительной записки оформлен грамотно, изложение последовательное.

Общий вывод:

В ходе квалификационной работы Гусев И.Е. разработал серверную часть информационной системы для автоматизации процессов согласования документов организации. Несмотря на частичную реализацию системы, Гусев И.Е. заслуживает присвоения квалификации бакалавра по направлению 09.03.04.

В работе не содержится информация с ограниченным доступом и отсутствуют сведения, представляющие коммерческую ценность.

Руководитель

канд. техн. наук

должность, уч. степень, звание

 16.06.25

подпись, дата

А. В. Фомин

инициалы, фамилия



Разработка программного комплекса для автоматизации процессов согласования документов

Гусев Иван Евгеньевич, 4132

Научный руководитель – Фомин А. В, канд. техн. наук

2025

Цель: Разработать платформу для автоматизации согласования документов в организации, что позволит решить проблему низкой эффективности и непрозрачности традиционного документооборота.

- *Проблема:
Существующие ручные и полуавтоматизированные процессы согласования документов приводят к задержкам, ошибкам и затрудняют контроль над бизнес-процессами.*
- *Методы и подходы:
Использование современных технологий разработки (Java Spring, JavaFX), принципов модульной архитектуры, а также анализ и адаптация лучших практик open-source систем автоматизации документооборота.*
- *Ожидаемые результаты:
Создание программного комплекса, который обеспечит сокращение времени согласования документов, повышение прозрачности процессов и возможность гибкой доработки под требования организации.*

1. Проанализировать существующие подходы и решения в области автоматизации согласования документов, выявить их преимущества и недостатки.
2. Сформулировать требования к функциональности и архитектуре платформы для автоматизации согласования документов.
3. Разработать архитектуру программного комплекса с использованием современных технологий (Java, Spring, Docker), обеспечивающую модульность и расширяемость системы.
4. Реализовать прототип платформы, включающий основные модули: управление пользователями, маршрутизация документов, система уведомлений и журналирование действий.
5. Провести тестирование разработанного программного комплекса и оценить его эффективность по ключевым показателям (скорость согласования, прозрачность, удобство использования).

- В современных организациях процессы согласования документов часто остаются неэффективными: ручные и устаревшие методы приводят к задержкам, ошибкам и усложняют контроль над документооборотом.
- Автоматизация согласования документов способствует повышению прозрачности, снижению административных издержек и минимизации человеческого фактора, что положительно сказывается на эффективности работы предприятий.
- Переход к электронному документообороту и активное внедрение open-source решений соответствуют современным тенденциям цифровизации бизнеса, а также требованиям по безопасности и гибкости ИТ-инфраструктуры.
- Разработка и внедрение таких систем особенно актуальны для компаний, стремящихся повысить конкурентоспособность, а также для ИТ-специалистов и руководителей, заинтересованных в оптимизации внутренних процессов.



Ключевые методы:

- Анализ существующих решений и требований к автоматизации документооборота
- Проектирование архитектуры программного комплекса
- Разработка и тестирование программного обеспечения

Подходы:

- Научный подход к анализу литературы и стандартов
- Практический подход к проектированию и реализации системы
- Модульный подход к построению архитектуры
- Инкрементальная разработка (итеративное добавление функционала)

Использованные технологии и инструменты:

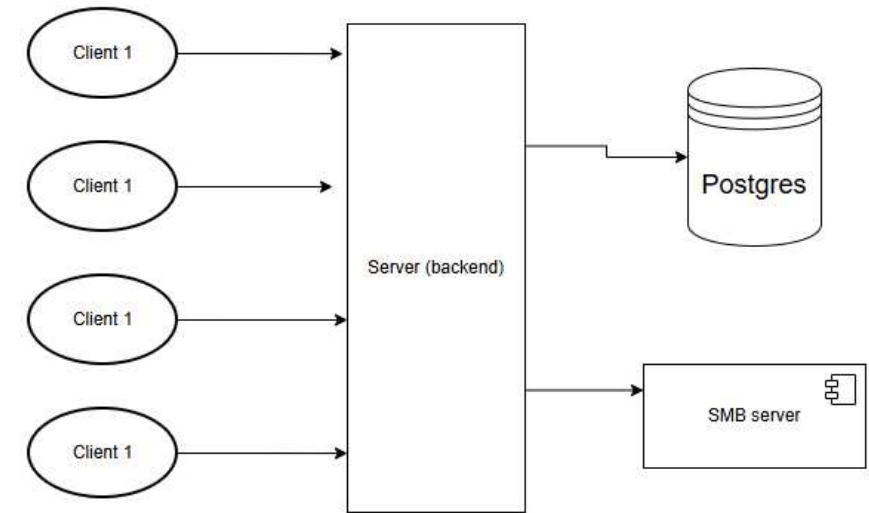
- Языки программирования: Java
- Фреймворки: Spring (backend), Spring + JavaFX (frontend)
- Базы данных: PostgreSQL
- Инструменты разработки: IntelliJ IDEA, Scene Builder
- Программное обеспечение: Docker (контейнеризация), Maven (сборка проекта)

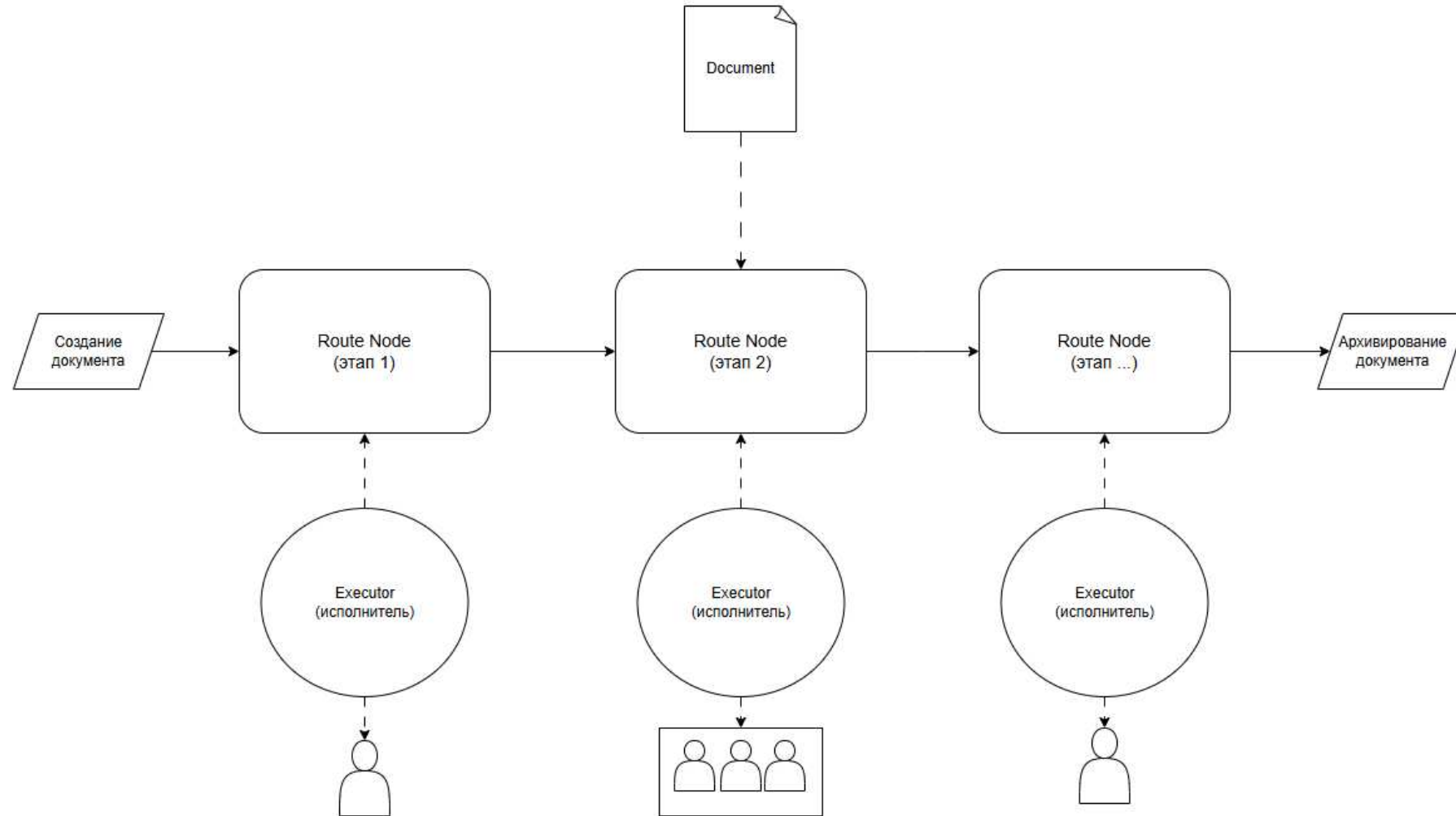


Архитектура: Проект построен по классической клиент-серверной архитектуре: клиенты взаимодействуют с сервером на Spring, который централизованно обрабатывает бизнес-логику, осуществляет работу с базой данных PostgreSQL и обеспечивает доступ к файловому хранилищу Samba.

Логика работы: Пользователь загружает документ и определяет маршрут согласования, система автоматически направляет документ ответственным лицам. Все действия фиксируются, а результаты согласования доступны в интерфейсе.

Безопасность: Все документы, загружаемые в систему, сохраняются на Samba-сервере в зашифрованном виде (алгоритм AES-256). При аутентификации система выдает JWT-токен.





- Разработана и внедрена платформа для автоматизации согласования документов.
- Реализована защищённая клиент-серверная архитектура с шифрованием файлов и аутентификацией пользователей.
- Система ускоряет процесс согласования, снижает количество ошибок и повышает прозрачность документооборота.
- Система реализована в виде модулей и может быть легко изменена под более сложные задачи.
- Решение готово к внедрению и дальнейшему развитию в организациях различного масштаба.

Сотрудники

Группы

Маршруты

Документы

Уведомления

Номер

7

Название

ШаблонДокумента.doc

Маршрут

Квальный Отчет

Этап

Проверка издержек

Редактор (id)

12

Дата создания

12.05.2024

Крайний срок

12.05.2025

Версия

2

Скачать документ

Прикрепить документ

Назад

Удалить

Сохранить

☐	Name	Image	Status	CPU (%)	Actions
☐	diploma-final		Running (3/3)	0.11%	
☐	postgres-1	postgres:16	Running	0.01%	
☐	samba-1	dperson/samba	Running	0%	
☐	app-1	diploma-final-app	Running	0.1%	

Auth

POST

/auth/register

Регистрация нового пользователя

POST

/auth/login

Вход в систему

Admin

GET

/users

Получить список пользователей

POST

/users

Создать пользователя

GET

/users/{id}

Получить пользователя по ID

PUT

/users/{id}

Обновить пользователя

DELETE

/users/{id}

Удалить пользователя

Documents

Workflow

GET

/routes

Получить список маршрутов

POST

/routes

Создать маршрут

GET

/routes/{id}

Получить маршрут

PUT

/routes/{id}

Обновить маршрут

DELETE

/routes/{id}

Удалить маршрут

Проверка маршрутизации документов (построение и изменение маршрутов).

- Тестирование отклонения и доработки документов.
- Контроль прав доступа для разных ролей (администратор, согласующий, наблюдатель).

Обработка ошибок:

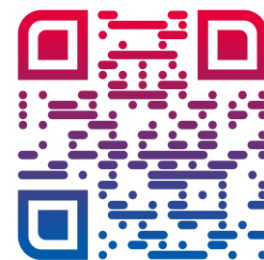
- Некорректные данные при вводе (неверные форматы файлов, пустые поля).
- Попытки несанкционированного доступа (JWT-валидация).

Безопасность и целостность:

- Шифрование файлов (AES-256) и контроль версий.
- Сохранность данных при передаче (HTTPS).

Вывод: решение обеспечивает надёжность, прозрачность и удобство автоматизации согласования документов в корпоративной системе.

- В ходе работы была успешно реализована платформа для автоматизации согласования документов, которая полностью соответствует поставленной цели и решает все ключевые задачи.
- Проект позволяет значительно повысить эффективность, прозрачность и безопасность процессов документооборота, что имеет важное значение для современных организаций.
- Разработанное решение может быть внедрено в компаниях различного масштаба с возможностью адаптации под конкретные бизнес-процессы. В дальнейшем можно расширить функциональность системы за счет интеграции с внешними сервисами, внедрения электронной подписи и создания мобильного клиента.
- Таким образом, разработанная платформа способствует цифровизации управления и открывает перспективы для дальнейших инноваций в области корпоративного документооборота



Спасибо за внимание!